

FraudGuard AI & GuardPay

Multi-Model Fraud Detection System

SYSTEM OVERVIEW, TERMINAL COMMANDS, ML MODELS & TESTING SCENARIOS

This manual details the design, technologies, and terminal instructions for the GuardPay payment portal and the FraudGuard AI security administration panel. The system incorporates real-time selection between three distinct Machine Learning models (Random Forest, Decision Tree, and Logistic Regression) integrated with rule-based heuristics to evaluate and block fraudulent transactions instantly.

Metadata:

Database: MySQL (Local Instance, service port: 3306)
Classifiers: Random Forest, Decision Tree, Logistic Regression
Stack: Python 3, Flask, Vanilla JS, Vanilla CSS3, HTML5
Manual Version: 1.2 (Multi-Model Support - June 2026)

1. Technical Stack & Architecture

The application is built using a clean, modern decoupled architecture consisting of a Python Flask backend, a native vanilla web interface, and a relational MySQL database. This allows fast, concurrent, and unbuffered evaluations.

Languages and Frontend Technologies

- **HTML5 (HyperText Markup Language):** Defines structures for the GuardPay Customer Portal and the FraudGuard AI Admin interface. Contains native elements and accessibility divisions.
- **Vanilla CSS3 (Cascading Style Sheets):** Styled custom component cards, flexboxes, and responsive grids with clean layouts, status badges, and slider elements.
- **JavaScript (ES6+ AJAX):** Controls forms, validates user entries, draws UI status notifications, performs fetch queries, and mocks browser fingerprinting via localStorage 'device_id'.

Backend and Frameworks

- **Python 3:** Powers the runtime server, executes database functions, and handles real-time machine learning predictions.
- **Flask:** Python micro-framework serving static front-end assets and hosting REST API routes (`/api/transactions``, `/api/security/settings``, etc.).
- **Cryptography (PBKDF2 Hashing):** Hashes client passwords dynamically using custom salting algorithms, storing only secure signatures in MySQL tables.

Database Layer

- **MySQL (Relational DB):** Maintains structural logging tables (users, transactions, user_devices, login_activity, fraud_results, and security_settings).
- **mysql-connector-python:** Database communications are routed through pure-python code ('use_pure=True') ensuring thread-safe concurrency.

2. Machine Learning Classifiers & Heuristics

Every transaction submitted goes through an evaluation process combining one of the three active machine learning models and a rule heuristics validator. Administrators can swap the active model from the Admin Panel in real-time.

Supported Machine Learning Models

- **1. Random Forest Classifier:** An ensemble of decision trees. It is highly accurate, resistant to overfitting, and ideal for capturing complex multi-dimensional fraud patterns.
- **2. Decision Tree Classifier:** A single hierarchical decision tree. Extremely fast, lightweight, and easy to interpret, but can overfit on outliers.
- **3. Logistic Regression Classifier:** A linear classification pipeline with standard feature scaling. Predicts probabilities using weighted coefficients. Best for identifying simpler linear boundaries.

Input Vector Feature Alignment

Each model receives an engineered vector consisting of: Amount, type indicator columns (PAYMENT, TRANSFER, CASH_OUT, DEBIT, CASH_IN), old balance, new balance, and the derived hour of transaction. The feature layout is pre-defined in 'ml/model_features.json'.

Rule-Based Heuristic Modifiers

Before and after ML predictions, rule heuristics run against the transaction context to inject additive penalty points:

- **Velocity Rule:** Blocks transactions if the user makes more than 2 payments in 60 seconds (adds a +50% risk penalty).
- **New Device Rule:** Adds a +25% risk penalty if the browser signature ('device_id') is not linked to the user in MySQL.
- **International Warnings:** Adds a +35% risk penalty if the transaction location is set to 'International'.
- **Global Blacklist:** Forces risk score to 100% and blocks the payment if the user ID or email is banned by the admin.

3. Terminal Execution & Startup Guide

Follow these exact step-by-step instructions in your terminal or PowerShell console to set up, train, and start the multi-model application:

Step 1: Install Required Libraries

Install the libraries needed for running the backend, training models, connecting to MySQL, and compiling PDFs:

```
pip install flask mysql-connector-python pandas scikit-learn joblib cryptography fpdf2
```

Step 2: Configure credentials (.env)

Verify the '.env' file in your root folder contains correct MySQL configuration:

```
# Example .env file
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=9342
DB_NAME=fraud_detection_db
SECRET_KEY=fraud_guard_ai_secret_key_123!@#
```

Step 3: Train All Classifiers

Run the model training pipeline. If the Kaggle dataset is missing, it will automatically generate synthetic transaction records, train all three models, and save them as pkl files:

```
python ml/train_model.py
```

Step 4: Seed Database Tables

Run the seeding script to initialize the MySQL tables, configure defaults, and insert seed login and user accounts:

```
python db_setup.py
```

Step 5: Start the Flask App

Launch the Flask server in the background. It will load all three models and start listening on port 5000:

```
python app.py
```

Step 6: Navigate to Gateways

- **Customer Gateway:** <http://127.0.0.1:5000/user.html> (user@test.com / user123)
- **Admin Dashboard:** <http://127.0.0.1:5000/admin.html> (admin@fraudguard.ai / admin123)

4. Verification & Testing Scenarios

To test the multi-model routing, open the Admin portal, choose a model, and submit payments in the Customer portal:

Scenario	Amount	Location	Device	Expected Outcome / Alert Triggered
1. Safe Payment	\$150.00	Local	Same	Approved (Green). Low risk. Balance debited in MySQL.
2. New Device	\$200.00	Local	New	Approved (Green, risk ~27%). Adds +25% device fee.
3. Velocity Block	\$50.00	Local	Same	Blocked (Red). Submit 3 times in 60s -> velocity penalty.
4. Large Amount	\$12,000.00	Local	Same	Blocked (Red). High amount anomaly flags transaction.
5. Int'l Payment	\$100.00	International	Same	Blocked (Red). Triggers international restriction rule.
6. Blacklist Match	Any	Local	Any	Blocked (Red). User ID banned in Admin -> 100% risk block.

How to Swapping Models dynamically:

- **1. Select Model:** Navigate to 'http://127.0.0.1:5000/security.html'. Locate the 'Active Fraud Model' dropdown, change selection (e.g. to Logistic Regression), and click 'Save Changes'.
- **2. Submit payment:** Navigate to 'user.html', input transaction values, and click 'Analyze Risk'.
- **3. Verify Routing:** Check the Admin server terminal. It prints 'ML Model (logistic_regression) risk probability' or similar, confirming routing. Go to 'transaction.html' ledger; the Model version column lists the active model name (e.g. LOGISTIC_REGRESSION_v1.0).